EXPERIMENT - 1

1. Title

Write a program to implement Caesar Cipher. Explain Caesar Cipher and the Modified Version of Caesar Cipher.

2. Outcome

Understand substitution-based encryption and decryption, implement Caesar Cipher, and explain how changing the shift key modifies the cipher.

3. Objectives

- To understand plaintext, ciphertext, encryption, decryption, and keys.
- To implement Caesar Cipher using a user-defined shift value.
- To understand the classical Caesar Cipher and its modified version.
- To test encryption and decryption using different input messages.

4. Nomenclature, Theory with Self-Assessment Questionnaire

4.1 Nomenclature

Term/Symbol	Meaning
Plaintext (P)	Original readable message
Ciphertext (C)	Encrypted message
K	Secret key or shift value
Encryption	Conversion of plaintext into ciphertext
Decryption	Conversion of ciphertext back into plaintext
Mod 26	Modulo operation used for the 26 English alphabet letters

4.2 Solution / Theory

Caesar Cipher is one of the simplest and oldest encryption techniques. It is a type of substitution cipher in which each letter of the plaintext is shifted by a fixed number of positions in the alphabet.

It is named after Julius Caesar, who reportedly used this technique to communicate securely.

For example, if the shift key is 3:

- A becomes D.
- B becomes E.

- C becomes F.
- X becomes A.
- Y becomes B.
- Z becomes C.

Thus, the plaintext HELLO becomes KHOOR.

The same key is used for encryption and decryption.

Encryption formula: \[

C=(P+K)\bmod 26 \]

Decryption formula: \[

P=(C-K+26)\bmod 26 \]

Where:

- \(P\) = numerical position of the plaintext letter.
- \(C\) = numerical position of the ciphertext letter.
- \(K\) = shift key.
- Modulo 26 ensures that the letters wrap around the alphabet.

The alphabet is represented numerically as A = 0, B = 1, C = 2, ..., Z = 25.

Example:

Plaintext: HELLO

Key: 3

Plaintext H E L L O

Shift by 3 K H O O R

Ciphertext: KHOOR

To decrypt, shift every ciphertext letter backward by three positions. The original message HELLO is obtained.

Modified Version of Caesar Cipher

A modified Caesar Cipher changes the traditional fixed shift of 3 by allowing the user to select a different shift key.

For example, if the key is 5:

- A becomes F.

- B becomes G.
- C becomes H.

The encryption formula remains:

\[C=(P+K)\bmod 26 \]

The decryption formula remains:

\[P=(C-K+26)\bmod 26 \]

The key can be any integer, with its effective shift determined modulo 26.

This modification makes the cipher more flexible than always using a fixed shift of 3. However, it does not provide strong security because there are only 26 possible shifts.

4.3 Algorithm

1. Start the program.
2. Read the plaintext message.
3. Read the shift key from the user.
4. Reduce the key modulo 26.
5. For every character in the message:
    - If it is an uppercase letter, shift it forward by the key.
    - If it is a lowercase letter, shift it forward by the key while preserving its case.
    - Leave spaces, digits, and punctuation unchanged.
6. Display the encrypted message.
7. Decrypt the ciphertext using the negative of the same key.
8. Display the decrypted message.
9. Stop.

4.4 Dependencies •

Python 3

- No external libraries are

required. **4.5 Code / Pseudocode** def

caesar_cipher(text, key):

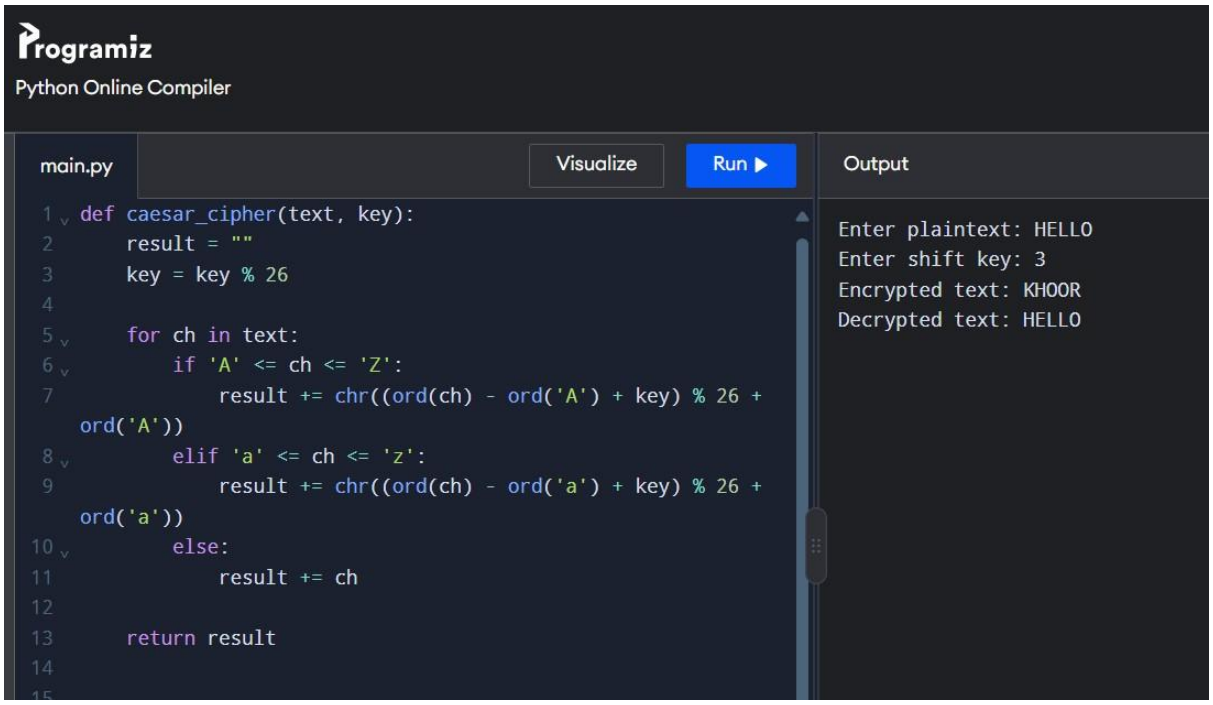
result = ""

key = key % 26

```
    for ch in text:
        if 'A' <= ch <= 'Z':
            result += chr((ord(ch) - ord('A') + key) % 26 + ord('A'))
elif 'a' <= ch <= 'z':
            result += chr((ord(ch) - ord('a') + key) % 26 + ord('a'))
        else:
            result += ch
    return result message = input("Enter
plaintext: ") key = int(input("Enter shift
key: ")) encrypted =
caesar_cipher(message, key) decrypted =
caesar_cipher(encrypted, -key)

print("Encrypted text:", encrypted) print("Decrypted
text:", decrypted)
```

4.6 Results



4.6.1 Test Cases

Input	Key	Expected Encrypted Output	Decrypted Output
HELLO	3	KHOOR	HELLO
Attack at dawn!	3	Dwwdfn dw gdzq!	Attack at dawn!
XYZ	3	ABC	XYZ
Hello	5	Mjqqt	Hello

4.6.2 Result Analysis

The program successfully encrypts plaintext by shifting alphabetic characters according to the specified key. Decryption reverses the shift and restores the original message.

Advantages

- Easy to understand and implement.
- Fast for short messages.
- Useful for understanding substitution ciphers.

Issues / Limitations

- Only 26 possible shifts exist.
- It is vulnerable to brute-force attacks.
- It preserves letter-frequency patterns.
- It is not secure for modern confidential communication.

4.7 References

1. Course notes on Cryptography and Information Security.
2. William Stallings, Cryptography and Network Security: Principles and Practice.

5. Lab Assignment

Modify the program to allow the user to choose between encryption and decryption. Test the program using shift keys 1, 13, and 25.

6. Quiz and Viva Questions

6.1 Quiz

Q1. Caesar Cipher is a:

(a) Transposition cipher (b) Substitution cipher (c) Hashing algorithm (d) Public-key algorithm

Answer: (b) Substitution cipher.

Q2. With key 3, A is encrypted as:

(a) B (b) C (c) D (d) Z

Answer: (c) D.

Q3. The number of possible Caesar shifts is:

(a) 2 (b) 10 (c) 26 (d) 128

Answer: (c) 26, including the ineffective zero shift.

Q4. Which formula represents Caesar Cipher encryption?

(a) $C=(P-K)\bmod26$ (b) $C=(P+K)\bmod26$ (c) $C=P\times K$ (d) $C=P\oplus K$

Answer: (b) $C=(P+K)\bmod26$. **6.2**

Viva Questions with Answers

Q1. What is cryptography?

Cryptography is the study of techniques used to protect information by transforming it into a form that unauthorized users cannot understand.

Q2. What is plaintext?

Plaintext is the original readable message before encryption.

Q3. What is ciphertext?

Ciphertext is the encrypted message obtained after applying an encryption algorithm.

Q4. What is Caesar Cipher?

Caesar Cipher is a substitution cipher that shifts each letter by a fixed number of positions in the alphabet.

Q5. What is a key?

A key is a value that controls the encryption and decryption process.

Q6. Why is modulo 26 used?

Modulo 26 is used because the English alphabet contains 26 letters. It allows the shift to wrap around from Z to A.

Q7. How is decryption performed?

Decryption is performed by subtracting the shift key from the ciphertext letter positions modulo 26.

Q8. What happens when the key is 26?

A shift of 26 is equivalent to a shift of zero, so the letters remain unchanged.

Q9. What is the modified Caesar Cipher used in this experiment?

It is a version in which the user can select the shift key instead of always using the traditional fixed shift of 3.

Q10. Is Caesar Cipher secure for modern communication?

No. It has a small keyspace and preserves letter-frequency patterns, making it easy to break.

EXPERIMENT - 2

1. Title

Write a program to implement the Simple Columnar Transposition technique.

2. Outcome

Understand transposition-based encryption and decryption and implement a technique that rearranges the positions of characters to encrypt a message.

3. Objectives

- To understand the difference between substitution and transposition ciphers.
- To arrange plaintext into rows and columns.
- To implement encryption using a column-order key.
- To implement decryption by reversing the column rearrangement.
- To understand padding and its use in completing an encryption grid.

4. Nomenclature, Theory with Self-Assessment Questionnaire 4.1

Nomenclature

Term	Meaning
Plaintext	Original readable message
Ciphertext	Encrypted message
Key / Column order	Order in which columns are read
Padding	Extra character added to complete an incomplete row
Transposition	Rearrangement of character positions without changing the characters themselves
Grid	Arrangement of plaintext into rows and columns

4.2 Solution / Theory

Simple Columnar Transposition is a classical encryption technique in which the characters of a plaintext message are rearranged according to a specific pattern.

Unlike Caesar Cipher, which replaces one letter with another, columnar transposition does not replace the characters. It changes only their positions.

In this technique, the plaintext is written row by row into a rectangular grid. The ciphertext is then generated by reading the columns from top to bottom in a particular order.

The column order acts as the encryption key.

Example

Consider the plaintext:

ATTACKATDAWN

Let the number of columns be 3.

Write the plaintext row by row into the grid:

Column 1	Column 2	Column 3
A	T	T
A	C	K
A	T	D
A	W	N

Now read each column from top to bottom.

- Column 1: AAAA
- Column 2: TCTW
- Column 3: TKDN

Therefore, the ciphertext is:

AAAATCTWTKDN

This example uses the natural column order 1, 2, 3. If a different key is used, the columns are read in the order specified by that key.

Encryption and Decryption

Encryption: Write plaintext into the grid row by row and read columns according to the key.

Decryption: Reconstruct the columns using the same key and read the grid row by row to recover the plaintext.

If the message length is not divisible by the number of columns, padding characters such as X can be added to complete the grid.

4.3 Algorithm

1. Start the program.
2. Read the plaintext message.
3. Read the column-order key from the user.
4. Validate that the key contains every column number exactly once.

5. Convert the plaintext into uppercase letters and remove non-alphabetic characters.
6. Add padding characters until the message length is divisible by the number of columns.
7. Arrange the padded plaintext into a grid row by row.
8. Read the columns from top to bottom according to the key.
9. Display the encrypted ciphertext.
10. For decryption, fill the columns in key order and read the reconstructed grid row by row.
11. Display the decrypted padded message.
12. Stop.

4.4 Dependencies •

Python 3

- No external libraries are

required. **4.5 Code / Pseudocode** def

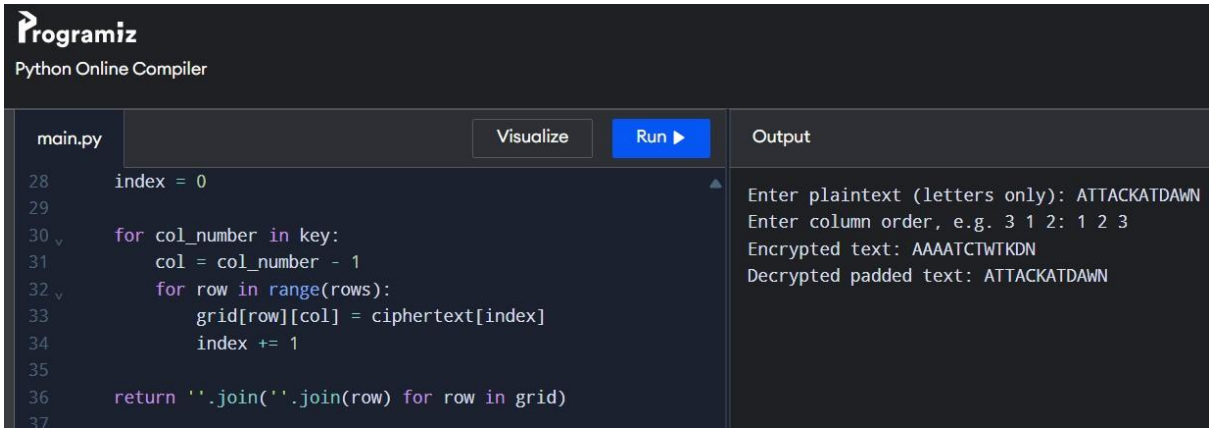
```
encrypt(plaintext, key):
    text = ''.join(ch for ch in plaintext.upper() if ch.isalpha())
cols = len(key)


    while len(text) % cols != 0:
        text += 'X'


    rows = len(text) // cols     grid = [
text[i:i + cols]         for i in range(0,
len(text), cols)
    ]
    ciphertext = ""
    for col_number in key:
        col = col_number - 1
for row in range(rows):
            ciphertext += grid[row][col]
```

```
    return ciphertext
def decrypt(ciphertext, key):     cols =
len(key)     rows = len(ciphertext) // cols
grid = [['] * cols for _ in range(rows)]
index = 0
    for col_number in key:
col = col_number - 1
for row in range(rows):
            grid[row][col] = ciphertext[index]
index += 1
    return ''.join(''.join(row) for row in grid)
message = input("Enter plaintext (letters only): ") key
= [
    int(x)     for x
in input(
        "Enter column order, e.g. 3 1 2: "
    ).split()
]
if sorted(key) != list(range(1, len(key) + 1)):
    print("Invalid key. Use each column number once.") else:
    encrypted = encrypt(message, key)     print("Encrypted
text:", encrypted)     print("Decrypted padded text:",
decrypt(encrypted, key))
```

4.6 Results



4.6.1 Test Cases

Input Plaintext	Key	Padded Grid Rows	Ciphertext
ATTACKATDAWN	1 2 3	ATT / ACK / ATD / AWN	AAAATCTWTKDN
HELLO	3 1 2	HEL / LOX	LXHLEO

For the second example, the grid is:

Column 1	Column 2	Column 3
H	E	L
L	O	X

Reading columns in the order 3, 1, 2 produces LXHLEO.

Decryption returns HELLOX. The final X is padding added to complete the grid.

4.6.2 Result Analysis

The program successfully encrypts the plaintext by rearranging character positions according to the specified column-order key.

Decryption reconstructs the original order of the characters. Any padding characters added during encryption may remain at the end of the decrypted message.

Advantages

- Simple to understand and implement.
- Preserves the original characters.
- Allows different column arrangements through a key.
- Useful for understanding transposition-based encryption.

Issues / Limitations

- Simple transposition alone is not secure by modern standards.
- Padding characters may remain visible after decryption.
- The sender and receiver must agree on the key and number of columns.
- Incorrect keys produce incorrect decryption.

4.7 References

1. Course notes on Cryptography and Information Security.
2. William Stallings, Cryptography and Network Security: Principles and Practice.
3. Python 3 documentation.

5. Quiz and Viva Questions

5.1 Quiz

Q1. Columnar Transposition changes:

(a) Character values (b) Character positions (c) Alphabet size (d) Encryption algorithm into a hash function

Answer: (b) Character positions.

Q2. The key in columnar transposition is generally:

(a) A column order (b) A prime number only (c) A plaintext letter (d) An IP address

Answer: (a) A column order.

Q3. Why is padding used?

(a) To change every letter (b) To complete the grid (c) To compress text (d) To generate a public key

Answer: (b) To complete the grid.

5.2 Viva Questions with Answers

Q1. What is transposition?

Transposition is an encryption technique in which the positions of characters are rearranged without changing the characters themselves.

Q2. How does columnar transposition work?

The plaintext is written row by row into a grid, and the ciphertext is generated by reading columns according to a key.

Q3. How is columnar transposition different from Caesar Cipher?

Caesar Cipher replaces each letter with another letter, whereas columnar transposition rearranges the positions of letters.

Q4. What is a transposition key?

It is the key that specifies the order in which columns are read during encryption.

Q5. Why must the key be a permutation?

Every column must be selected exactly once. Repeated or missing column numbers could result in incorrect encryption or decryption.

Q6. What is padding?

Padding consists of extra characters added to complete the final row of the grid.

Q7. Can the original message contain repeated letters?

Yes. Repeated letters are retained, but their positions change during encryption.

Q8. What information must the sender and receiver share?

They must share the column-order key, the number of columns, and the padding convention.

Q9. Does transposition change letter frequencies?

No. It preserves the number of occurrences of each character.

Q10. Is simple columnar transposition secure on its own?

No. It is a classical educational cipher and should not be used alone to protect sensitive modern communications.

EXPERIMENT - 3

1. Title

Write a program to implement the Rail Fence technique. Explain the Rail Fence technique with an example.

2. Outcome

Understand zigzag transposition and implement encryption and decryption using a selected number of rails.

3. Objectives

- To understand the Rail Fence cipher as a transposition cipher.
- To arrange plaintext diagonally in a zigzag pattern.
- To generate ciphertext by reading the rails row by row.
- To implement encryption and decryption using Python.
- To understand the importance of the number of rails.

4. Nomenclature, Theory with Self-Assessment Questionnaire

4.1 Nomenclature

Term	Meaning
Plaintext	Original readable message
Ciphertext	Encrypted message
Rail	A horizontal row in the zigzag arrangement
Number of rails / Depth	Number of rows used for encryption
Transposition	Rearrangement of character positions
Encryption	Converting plaintext into ciphertext
Decryption	Recovering plaintext from ciphertext

4.2 Solution / Theory

Rail Fence Cipher is a classical transposition cipher in which the plaintext is written diagonally in a zigzag pattern across a specified number of rows, called rails.

After all characters are arranged, the characters are read row by row, from the first rail to the last rail, to generate the ciphertext.

The number of rails acts as the key.

Example

Consider the plaintext:

WEAREDISCOVEREDFLEEATONCE

Number of rails: 3

Write the message in a zigzag pattern across three rails:

W . . . E . . . C . . . R . . . L . . . T . . .

. E . R . D . S . O . E . E . F . E . A . O . C .

. . A . . . I . . . V . . . D . . . E . . . N . .

The characters are placed diagonally downward until the bottom rail is reached, and then diagonally upward until the top rail is reached. This pattern repeats until all characters are written.

Now read each rail from left to right.

- First rail: WECRLTE
- Second rail: ERDSOEEFEAOC
- Third rail: AIVDEN

Combining the rails gives the ciphertext:

WECRLTEERDSOEEFEAOCAIVDEN

Encryption and Decryption

Encryption:

1. Write the plaintext in a zigzag pattern across the selected number of rails.
2. Read each rail from left to right.
3. Combine the characters to produce the ciphertext.

Decryption:

1. Recreate the zigzag path for the ciphertext length.
2. Determine how many characters belong to each rail.
3. Fill the rails with the ciphertext characters in order.
4. Read the characters along the zigzag path to recover the plaintext.

4.3 Algorithm

1. Start the program.
2. Read the plaintext message.
3. Read the number of rails from the user.
4. Create an empty string for each rail.

5. Begin at the first rail and move downward through the rails.
6. When the last rail is reached, reverse direction and move upward.
7. Continue this process until every character is placed.
8. Concatenate the rail strings from top to bottom to obtain the ciphertext.
9. For decryption, recreate the zigzag path and count the positions belonging to each rail.
10. Fill each rail with the corresponding ciphertext characters.
11. Read along the zigzag path to reconstruct the original message.
12. Display the encrypted and decrypted messages.
13. Stop.

4.4 Dependencies •

Python 3

- No external libraries are

required. **4.5 Code / Pseudocode** def

encrypt_rail_fence(text, rails): if rails

<= 1: return text

fence = [''] * rails

row = 0 direction

= 1

for ch in text:

fence[row] += ch

if row == 0:

direction = 1 elif

row == rails - 1:

direction = -1

row += direction

return ''.join(fence)

```
def decrypt_rail_fence(ciphertext, rails):
if rails <= 1:        return ciphertext
   path = []
row = 0
direction = 1

   for _ in ciphertext:
path.append(row)

      if row == 0:
direction = 1        elif
row == rails - 1:
direction = -1
      row += direction

   counts = [
path.count(r)        for r
in range(rails)
   ]

   fence = []
index = 0

   for count in counts:        fence.append(
list(ciphertext[index:index + count])
      )
      index += count
```

```
    positions = [0] * rails
result = ""

    for r in path:
        result += fence[r][positions[r]]
positions[r] += 1

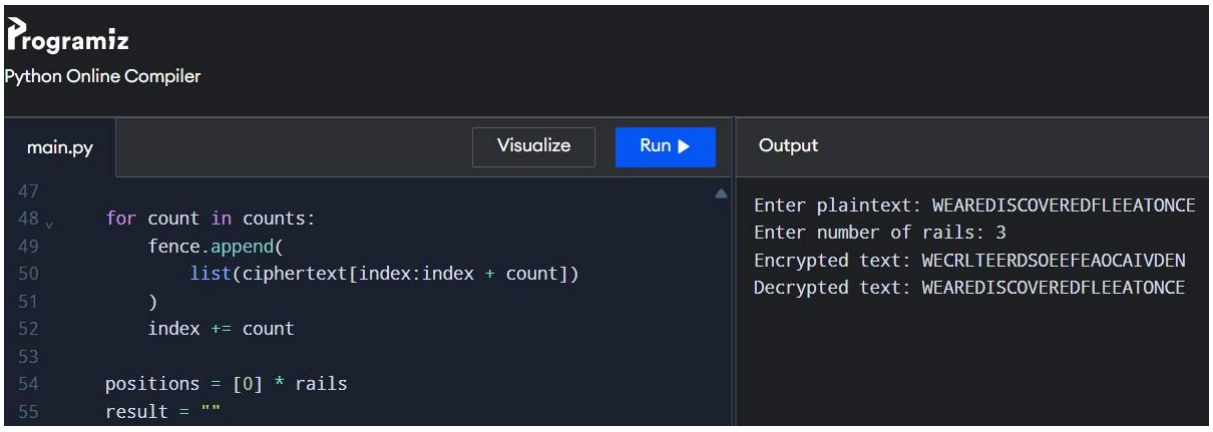
    return result

message = input("Enter plaintext: ") rails
= int(input("Enter number of rails: "))

encrypted = encrypt_rail_fence(message, rails)

print("Encrypted text:", encrypted) print("Decrypted text:",
decrypt_rail_fence(encrypted, rails))
```

4.6 Results



4.6.1 Test Cases

Plaintext	Expected Ciphertext
WEAREDISCOVEREDFLEEATONCE 3	WECRLTEERDSOEEFEAOCAIVDEN

HELLOWORLD	2	HLOOLELWRD

For the second example, using two rails:

H . L . O . O . L .

. E . L . W . R . D

Reading the first rail gives HLOOL.

Reading the second rail gives ELWRD.

Therefore, the ciphertext is:

HLOOLELWRD

Running the decryption function with the same number of rails should reproduce the original plaintext.

4.6.2 Result Analysis

The program successfully encrypts the plaintext by arranging its characters in a zigzag pattern and reading the rails sequentially.

The decryption process reconstructs the zigzag arrangement and restores the original message.

Advantages

- Easy to understand and implement.
- Demonstrates the concept of transposition encryption.
- Does not require alphabet substitution.
- Uses the number of rails as a simple encryption key.

Issues / Limitations

- The pattern can be discovered through trial and error.
- The number of rails must be known or guessed.
- Short messages are particularly easy to analyse.
- It is not suitable for modern security on its own.

4.7 References

1. Course notes on Cryptography and Information Security.
2. William Stallings, Cryptography and Network Security: Principles and Practice.
3. Python 3 documentation.

5. Quiz and Viva Questions

5.1 Quiz

Q1. Rail Fence is a:

(a) Substitution cipher (b) Transposition cipher (c) Hash function (d) Compression algorithm

Answer: (b) Transposition cipher.

Q2. In Rail Fence Cipher, plaintext is written in:

(a) A zigzag pattern (b) Reverse alphabetical order (c) Binary format only (d) A circular pattern

Answer: (a) A zigzag pattern.

Q3. To decrypt a Rail Fence cipher, one must know or determine the:

(a) Rail count (b) CPU speed (c) File extension (d) Network port

Answer: (a) Rail count.

5.2 Viva Questions with Answers

Q1. What is the Rail Fence technique?

Rail Fence is a transposition cipher that writes plaintext in a zigzag pattern across multiple rails and reads the rails row by row to create ciphertext.

Q2. Why is it called Rail Fence?

It is called Rail Fence because the zigzag arrangement resembles characters written across horizontal rails.

Q3. What is the key in Rail Fence Cipher?

The key is the number of rails used to arrange the plaintext.

Q4. How is encryption performed?

The plaintext is written in a downward and upward zigzag pattern, and the characters are concatenated rail by rail.

Q5. How is decryption performed?

The zigzag path is reconstructed, the ciphertext characters are placed into their respective rails, and the plaintext is recovered by following the zigzag path.

Q6. Does Rail Fence substitute letters?

No. It only rearranges the positions of the characters.

Q7. What happens when only one rail is used?

The text remains unchanged because there is only one row and no rearrangement occurs.

Q8. What happens if the rail count is greater than the message length?

Some rails may remain empty, and the cipher may provide little or no meaningful rearrangement.

Q9. How does Rail Fence differ from columnar transposition?

Rail Fence arranges characters in a zigzag pattern, whereas columnar transposition writes characters row by row into a grid and reads the columns according to a key.

Q10. Is Rail Fence secure for modern communication?

No. It is a classical cipher that can be broken by analysing the pattern or trying possible rail counts.

EXPERIMENT - 4

1. Title

Write a program to implement the Homophonic Substitution technique.

2. Outcome

Understand the Homophonic Substitution technique and implement encryption and decryption using multiple ciphertext symbols for a single plaintext character.

3. Objectives

- To understand the concept of homophonic substitution.
- To learn how multiple ciphertext symbols can represent one plaintext character.
- To implement encryption and decryption using a homophonic substitution mapping.
- To understand how this technique reduces the usefulness of ordinary letter-frequency analysis.

4. Nomenclature, Theory with Self-Assessment Questionnaire

4.1 Nomenclature Term /

Symbol	Meaning
Plaintext	Original readable message
Ciphertext	Encrypted message
Substitution	Replacing a plaintext character with another symbol
Homophone	One of multiple symbols representing the same plaintext character
Encryption	Converting plaintext into ciphertext
Decryption	Recovering plaintext from ciphertext
Substitution key	Mapping that defines the symbols associated with each plaintext character
Frequency analysis	Analysing how often symbols appear to identify patterns in encrypted text

4.2 Solution / Theory

Homophonic Substitution Cipher is a classical substitution cipher in which a single plaintext character can be represented by multiple possible ciphertext symbols.

In a simple substitution cipher, each plaintext letter is consistently replaced by one ciphertext letter. In a homophonic substitution cipher, one plaintext letter may have several possible ciphertext representations.

For example, the plaintext letter E may be represented by 12, 25, or 37. Whenever the letter E appears, the encryption process can select one of these symbols.

This technique is particularly useful for reducing the effectiveness of simple frequency analysis. In ordinary English text, certain letters, such as E and T, occur more frequently than others. If every E were always represented by the same symbol, that symbol would also appear frequently in the ciphertext.

By assigning multiple symbols to common plaintext letters, the frequency of occurrences can be distributed across several ciphertext symbols.

Example mapping:

Plaintext Character	Possible Ciphertext Symbols
A	11, 21
B	12
C	13
E	14, 24, 34
H	15
L	16, 26
O	17, 27

For the plaintext HELLO, one possible encryption is:

- H → 15
- E → 24
- L → 16
- L → 26
- O → 17

Ciphertext: 15 24 16 26 17

The same plaintext character can produce different ciphertext symbols on different occurrences. For example, E can be encrypted as either 14, 24, or 34.

During decryption, each ciphertext symbol is mapped back to its corresponding plaintext character.

Important: The example mapping above is illustrative and incomplete. A complete implementation needs a mapping for every supported plaintext character.

4.3 Algorithm

Encryption

1. Start the program.
2. Define a homophonic substitution mapping.
3. Read the plaintext message.
4. Convert the message to uppercase.
5. For each character, check the mapping.
6. If the character has multiple possible ciphertext symbols, select one randomly.
7. Append the selected symbol to the ciphertext.
8. Display the ciphertext.

Decryption

1. Read the ciphertext symbols.
2. Define the reverse mapping from ciphertext symbols to plaintext characters.
3. Separate the ciphertext into individual symbols.
4. Replace each symbol with its corresponding plaintext character.
5. Display the decrypted message.
6. Stop.

4.4 Dependencies

- Python 3
- random module, included in Python's standard library **4.5 Code /**

Pseudocode import random

```
key = {
    'A': ['11', '21'],
'B': ['12'],
    'C': ['13', '23'],
    'D': ['14', '24'],
    'E': ['15', '25', '35'],
    'F': ['16'],
```

```
    'G': ['17'],
    'H': ['18', '28'],
    'I': ['19', '29'],
    'J': ['31'],
    'K': ['32'],
    'L': ['33', '43'],
    'M': ['34'],
    'N': ['36', '46'],
    'O': ['37', '47'],
    'P': ['38'],
    'Q': ['39'],
    'R': ['41', '51'],
    'S': ['42', '52'],
    'T': ['44', '54'],
    'U': ['45'],
    'V': ['48'],
    'W': ['49'],
    'X': ['53'],
    'Y': ['55'],
    'Z': ['56']
}

reverse_key = {}

for letter, symbols in key.items():
for symbol in symbols:
reverse_key[symbol] = letter
message = input("Enter plaintext:
").upper()
```

```
ciphertext = []

for ch in message:     if ch in key:
ciphertext.append(random.choice(key[ch]))
   elif ch == ' ':
       ciphertext.append('/')

encrypted = ' '.join(ciphertext)

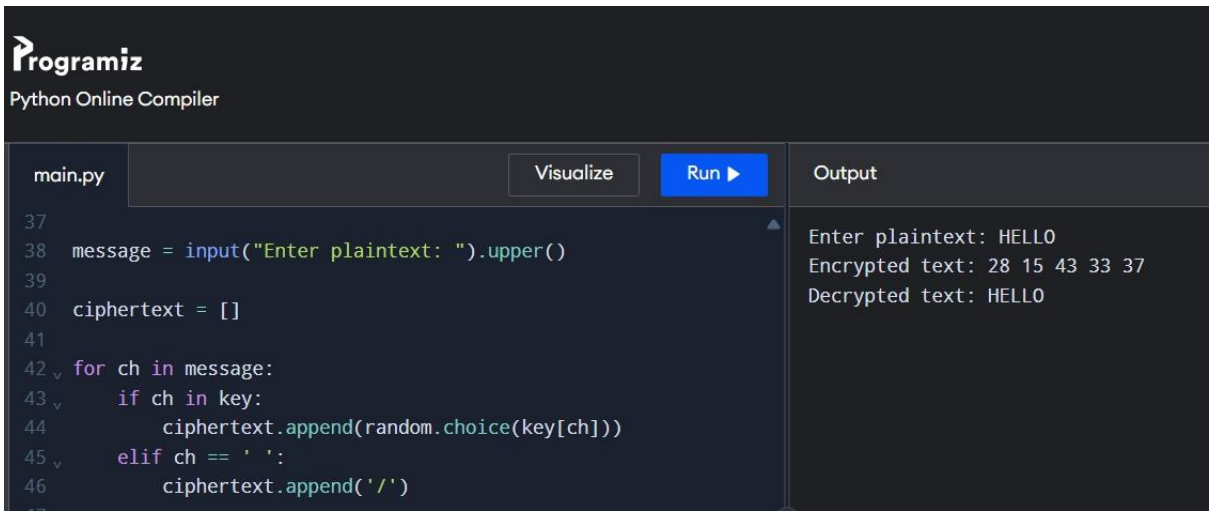
print("Encrypted text:", encrypted)

parts = encrypted.split() decrypted
= ""

for symbol in parts:     if
symbol == '/':         decrypted
+= ' '     elif symbol in
reverse_key:
       decrypted += reverse_key[symbol]

print("Decrypted text:", decrypted)
```

4.6 Results



4.6.1 Test Cases

Since encryption uses random selection, the ciphertext can differ between executions.

Input Plaintext	Possible Ciphertext	Decrypted Output
HELLO	18 25 33 43 37	HELLO
EEE	15 35 25	EEE
ABC	21 12 23	ABC
CAT	13 11 54	CAT

4.6.2 Result Analysis

The program successfully implements homophonic substitution by allowing certain plaintext letters to have multiple possible ciphertext symbols.

Random selection distributes occurrences of the same letter among its available symbols. During decryption, the reverse mapping converts each symbol back into the original plaintext character.

Advantages

- Reduces the effectiveness of simple frequency analysis.
- Allows multiple ciphertext representations for common letters.
- Easy to demonstrate using a dictionary-based implementation.
- Helps explain the relationship between plaintext frequencies and ciphertext patterns.

Issues / Limitations

- Security depends on the design of the substitution mapping.
- A small or poorly designed mapping can reveal patterns.

- The mapping must be shared securely between the sender and receiver.
- It is not suitable for protecting sensitive modern communications.

4.7 References

1. Course notes on Cryptography and Information Security.
2. William Stallings, Cryptography and Network Security: Principles and Practice.
3. Python 3 documentation: https://docs.python.org/3/

5. Quiz and Viva Questions

5.1 Quiz

Q1. Homophonic substitution allows:

(a) One plaintext character to have multiple ciphertext representations (b) Only numbers to be encrypted (c) Plaintext to remain unchanged (d) Characters to be deleted Answer: (a) One plaintext character to have multiple ciphertext representations.

Q2. The primary purpose of homophonic substitution is to:

(a) Compress a message (b) Reduce the effectiveness of frequency analysis (c) Increase message length only (d) Generate a digital signature Answer: (b) Reduce the effectiveness of frequency analysis.

Q3. In homophonic substitution, the same plaintext letter:

(a) Must always produce the same symbol (b) Can produce different ciphertext symbols (c) Cannot be encrypted (d) Must be removed

Answer: (b) Can produce different ciphertext symbols.

Q4. Decryption requires:

(a) The reverse substitution mapping (b) Only the plaintext (c) A sorting algorithm (d) A compression dictionary only

Answer: (a) The reverse substitution mapping.

5.2 Viva Questions with Answers

Q1. What is a Homophonic Substitution Cipher?

It is a substitution cipher in which one plaintext character can be represented by multiple ciphertext symbols.

Q2. What is a homophone in cryptography?

A homophone is one of several possible ciphertext symbols that represent the same plaintext character.

Q3. How is homophonic substitution different from simple substitution?

In simple substitution, each plaintext character consistently maps to one ciphertext symbol. In homophonic substitution, a plaintext character may map to multiple symbols.

Q4. Why is homophonic substitution used?

It is used to reduce the usefulness of ordinary letter-frequency analysis by distributing occurrences of common letters among multiple ciphertext symbols.

Q5. What is frequency analysis?

Frequency analysis is a cryptanalysis technique that studies how often characters or symbols occur in ciphertext to identify possible plaintext patterns.

Q6. Why can the same letter produce different ciphertext symbols?

Because the encryption algorithm selects one symbol from the set assigned to that plaintext letter.

Q7. How is decryption performed?

Each ciphertext symbol is looked up in the reverse mapping to recover its corresponding plaintext character.

Q8. Why is a reverse mapping necessary?

It allows the receiver to identify which plaintext character each ciphertext symbol represents.

Q9. What is the role of randomness in this implementation?

Randomness selects among the available ciphertext symbols for a plaintext character that has multiple representations.

Q10. Is homophonic substitution completely secure?

No. It can reduce simple frequency-analysis clues, but it remains vulnerable to other forms of cryptanalysis and is not suitable for modern security by itself.

EXPERIMENT - 5

1. Title

Write a program to implement the Polygram Substitution Cipher.

2. Outcome

Understand polygram substitution and implement encryption and decryption in which groups of plaintext characters are substituted with corresponding groups of ciphertext characters.

3. Objectives

- To understand the concept of polygram substitution.
- To learn how groups of characters are encrypted together.
- To implement encryption and decryption using a predefined substitution mapping.
- To understand the difference between monoalphabetic and polygram substitution.
- To test the program with sample plaintext and ciphertext.

4. Nomenclature, Theory with Self-Assessment Questionnaire

4.1 Nomenclature

Term / Symbol	Meaning
Plaintext	Original readable message
Ciphertext	Encrypted message
Polygram	A group of two or more characters
Substitution	Replacing a plaintext group with another group
Encryption	Converting plaintext into ciphertext
Decryption	Recovering plaintext from ciphertext
Key	Mapping that defines the substitutions
Digraph	A group of two characters
Trigraph	A group of three characters

4.2 Solution / Theory

Polygram Substitution Cipher is a classical encryption technique in which groups of characters are substituted rather than encrypting each character independently.

The word polygram refers to a group of multiple characters. Depending on the cipher, these groups can contain two characters, three characters, or more.

For example, a digraph substitution cipher operates on pairs of letters, while a trigraph substitution cipher operates on groups of three letters.

In a simple monoalphabetic substitution cipher, each plaintext letter is replaced independently by another symbol. In polygram substitution, a group of letters is treated as a single unit and replaced with another group according to a predefined mapping.

For this experiment, a simple digraph substitution mapping will be used to demonstrate the principle.

Example substitution mapping: **Plaintext Pair** **Ciphertext Pair**

HE	XY
LL	AB
OW	CD
AB	EF
CD	GH
HI	JK

Consider the plaintext:

HELLOW

Divide it into pairs:

HE LL OW

Apply the substitution mapping:

- HE → XY
- LL → AB
- OW → CD

Therefore, the ciphertext is:

XYABCD

During decryption, each ciphertext pair is mapped back to its corresponding plaintext pair.

Important: The mapping in this experiment is illustrative and is not a standard secure cipher. It demonstrates the principle of polygram substitution using fixed two-character groups.

If the plaintext length is odd, a padding character such as X can be added to complete the final pair. In this experiment, the program will use X padding when necessary.

4.3 Algorithm

Encryption

1. Start the program.
2. Define a dictionary containing plaintext-pair and ciphertext-pair mappings.
3. Read the plaintext message.
4. Convert the message to uppercase and remove spaces.
5. If the message length is odd, append X.
6. Divide the message into pairs of characters.
7. Replace each plaintext pair using the substitution mapping.
8. Display the ciphertext.

Decryption

1. Read the ciphertext.
2. Construct the reverse substitution mapping.
3. Divide the ciphertext into pairs of characters.
4. Replace each ciphertext pair with its corresponding plaintext pair.
5. Display the decrypted message.
6. Stop.

4.3.1 Dependencies •

Python 3

- No external libraries are

required. **4.4 Code / Pseudocode** def

encrypt(text, key):

```
    text = ''.join(text.upper().split())

    if len(text) % 2 != 0:
        text += 'X'

    result = ""
```

```
    for i in range(0, len(text), 2):
pair = text[i:i + 2]         result
+= key.get(pair, pair)

    return result



def decrypt(text, key):     reverse_key =
{         value: key_name         for
key_name, value in key.items()
    }

    result = ""

    for i in range(0, len(text), 2):
        pair = text[i:i + 2]         result +=
reverse_key.get(pair, pair)

    return result



key = {
    'HE': 'XY',
    'LL': 'MN',
    'OW': 'PQ',
    'AB': 'EF',
    'CD': 'GH',
    'HI': 'JK'
}
```

message = input("Enter plaintext: ")

encrypted = encrypt(message, key) print("Encrypted text:", encrypted)

decrypted = decrypt(encrypted, key) print("Decrypted text:", decrypted)

4.5 Results



4.5.1 Test Cases

Input Plaintext Ciphertext Decrypted Output

HELLOW XYMNPQ HELLOW

HELLO XYMN OX HELLOX

ABCD EFGH ABCD

HIAB JKEF HIAB

Note: Spaces are removed before encryption. If the original message has an odd number of characters, the final X is padding. It may remain after decryption and can be removed if it was added only for padding.

4.5.2 Result Analysis

The program successfully demonstrates polygram substitution by processing the plaintext in groups of two characters.

Each mapped pair is replaced by its corresponding ciphertext pair. During decryption, the reverse mapping restores the original pairs. Unmapped pairs remain unchanged, making this implementation suitable for demonstrating the basic principle.

Advantages

- Encrypts groups of characters rather than individual characters.
- Demonstrates the basic concept of digraph substitution.
- Easy to implement using dictionaries.
- Can be extended to larger groups of characters.

Issues / Limitations

- The security depends on the substitution mapping.
- A small or predictable mapping is easy to analyse.
- Unmapped pairs remain unchanged in this educational implementation.
- Padding may remain after decryption.
- It is not suitable for modern secure communication.

4.6 References

1. Course notes on Cryptography and Information Security.
2. William Stallings, Cryptography and Network Security: Principles and Practice.
3. Python 3 documentation: https://docs.python.org/3/

5. Quiz and Viva Questions

5.1 Quiz

Q1. Polygram substitution encrypts:

(a) Only individual characters (b) Groups of characters (c) Only numbers (d) Entire files without changing their contents Answer: (b) Groups of characters.

Q2. A digraph consists of:

(a) One character (b) Two characters (c) Three characters (d) Four characters

Answer: (b) Two characters.

Q3. A trigraph consists of:

(a) One character (b) Two characters (c) Three characters (d) Five characters

Answer: (c) Three characters.

Q4. What is the purpose of a substitution mapping?

(a) To define how plaintext groups are replaced with ciphertext groups (b) To calculate the message length only (c) To delete repeated characters (d) To sort the plaintext alphabetically

Answer: (a) To define how plaintext groups are replaced with ciphertext groups.

Q5. Why might padding be required?

(a) To make the message readable (b) To complete the final group when the plaintext length is odd (c) To remove the key (d) To convert ciphertext to binary

Answer: (b) To complete the final group when the plaintext length is odd.

5.2 Viva Questions with Answers

Q1. What is a Polygram Substitution Cipher?

A Polygram Substitution Cipher is an encryption technique in which groups of characters are replaced by other groups according to a predefined substitution mapping.

Q2. What is a polygram?

A polygram is a group of multiple characters treated as a unit during encryption.

Q3. What is a digraph?

A digraph is a group of two characters.

Q4. What is a trigraph?

A trigraph is a group of three characters.

Q5. How does polygram substitution differ from simple substitution?

Simple substitution replaces each character independently, whereas polygram substitution replaces groups of characters together.

Q6. What is the role of a key in polygram substitution?

The key defines the mapping between plaintext groups and ciphertext groups.

Q7. How is encryption performed in this experiment?

The plaintext is divided into pairs of characters, and each pair is replaced according to the predefined substitution dictionary.

Q8. How is decryption performed?

A reverse mapping is constructed, and each ciphertext pair is replaced by its corresponding plaintext pair.

Q9. Why is padding used?

Padding completes the final group when the plaintext contains an odd number of characters.

Q10. What happens if a pair is not present in the substitution mapping?

In this implementation, the pair remains unchanged.

Q11. Can polygram substitution use groups larger than two characters?

Yes. It can use groups of three or more characters, depending on the cipher design.

Q12. Is polygram substitution secure for modern communication?

Not necessarily. Security depends on the cipher design and key. A small, predictable substitution mapping such as the one used in this experiment is not secure for modern confidential communication.

EXPERIMENT - 6

1. Title

Write a program to implement Polyalphabetic Substitution Cipher.

2. Outcome

Understand polyalphabetic substitution and implement encryption and decryption using multiple substitution alphabets controlled by a key.

3. Objectives

- To understand the concept of polyalphabetic substitution.
- To learn how multiple substitution alphabets are used for encryption.
- To implement encryption and decryption using the Vigenère Cipher.
- To understand the role of a repeating key in the encryption process.
- To analyse the advantages and limitations of polyalphabetic substitution.

4. Nomenclature, Theory with Self-Assessment Questionnaire

4.1 Nomenclature

Term / Symbol	Meaning
Plaintext (P)	Original readable message
Ciphertext (C)	Encrypted message
Key (K)	Word or sequence controlling the substitutions
Encryption	Conversion of plaintext into ciphertext
Decryption	Recovery of plaintext from ciphertext
Polyalphabetic cipher	Cipher that uses multiple substitution alphabets
Vigenère Cipher	A classical polyalphabetic substitution cipher
Modulo 26	Operation used to wrap alphabet positions within 26 letters

4.2 Solution / Theory

Polyalphabetic Substitution Cipher is an encryption technique that uses multiple substitution alphabets instead of a single fixed substitution alphabet.

In a monoalphabetic substitution cipher, each plaintext letter is always replaced by the same ciphertext letter. In a polyalphabetic substitution cipher, the replacement can change depending on the corresponding key character.

One of the most well-known examples is the Vigenère Cipher. It uses a keyword that is repeated until it matches the length of the plaintext.

Each letter is converted into a numerical value:

A = 0, B = 1, C = 2, ..., Z = 25.

Encryption formula: \[

C_i=(P_i+K_i)\bmod 26 \]

Decryption formula:

\[P_i=(C_i-K_i+26)\bmod 26 \]

Where:

- \(P_i\) represents the plaintext letter's numerical value.
- \(C_i\) represents the ciphertext letter's numerical value.
- \(K_i\) represents the key letter's numerical value.
- \(i\) represents the position of a character.

Example

Plaintext: HELLO

Key: KEYKE

Plaintext H E L L O

Key K E Y K E

Ciphertext R I J V S

Therefore, the ciphertext is:

RIJVS

For example, the first character is encrypted as follows:

- H = 7
- K = 10
- (7 + 10) mod 26 = 17
- 17 corresponds to R.

During decryption, the key value is subtracted from each ciphertext character to recover the original plaintext.

Polyalphabetic substitution makes simple frequency analysis more difficult than in a basic monoalphabetic substitution cipher. However, classical implementations with repeating keys remain vulnerable to cryptanalysis.

4.3 Algorithm

Encryption

1. Start the program.
2. Read the plaintext message.
3. Read the keyword.
4. Convert the plaintext and key to uppercase.
5. Repeat the key until it matches the number of alphabetic characters in the plaintext.
6. Convert each plaintext and key character into a numerical value.
7. Add the values and apply modulo 26.
8. Convert the result back into a letter.
9. Preserve spaces and punctuation.
10. Display the ciphertext.

Decryption

1. Read the ciphertext and key.
2. Convert the characters into numerical values.
3. Subtract the key value from the ciphertext value.
4. Apply modulo 26.
5. Convert the result back into letters.
6. Display the decrypted message.
7. Stop.

4.4 Dependencies •

Python 3

- No external libraries are

required. **4.5 Code / Pseudocode** def

encrypt(text, key): result = "" key =

key.upper()

j = 0

```
    for ch in text.upper():
if ch.isalpha():
            p = ord(ch) - ord('A')            k =
ord(key[j % len(key)]) - ord('A')
result += chr((p + k) % 26 + ord('A'))
j += 1         else:
            result += ch

    return result


def decrypt(text, key):
result = ""     key =
key.upper()
    j = 0

    for ch in text.upper():
if ch.isalpha():
            c = ord(ch) - ord('A')            k =
ord(key[j % len(key)]) - ord('A')
result += chr((c - k) % 26 + ord('A'))
j += 1         else:
            result += ch

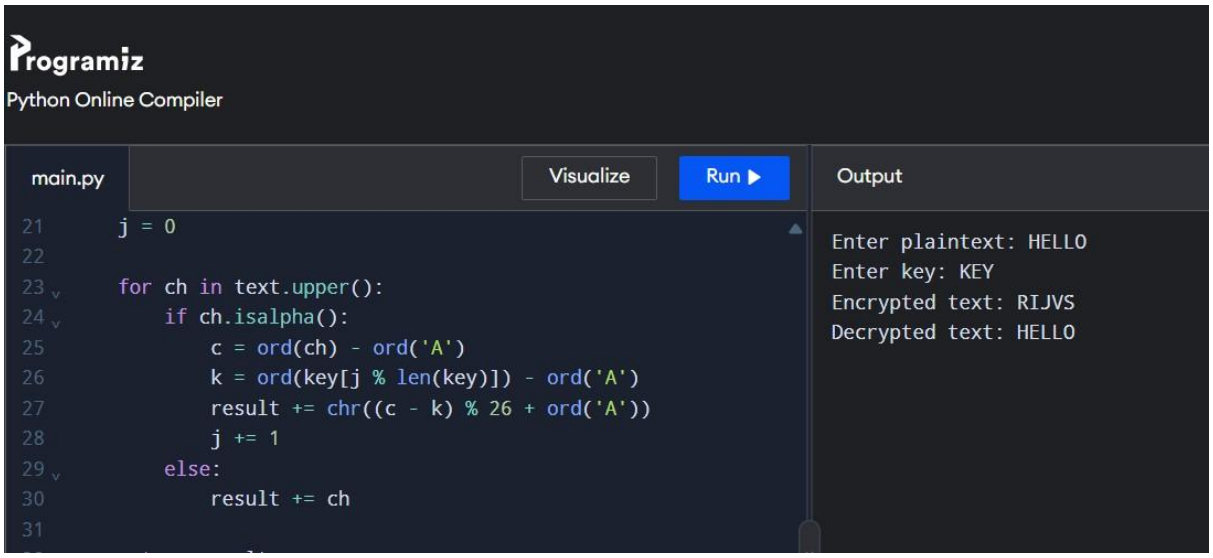
    return result


message = input("Enter plaintext: ") key
= input("Enter key: ").strip()
```

```
if not key.isalpha():
    print("Key must contain letters only.") else:
    encrypted = encrypt(message, key)
print("Encrypted text:", encrypted)

    decrypted = decrypt(encrypted, key)
print("Decrypted text:", decrypted)
```

4.6 Results



4.6.1 Test Cases

Input Plaintext	Key	Expected Ciphertext	Decrypted Output
HELLO	KEY	RIJVS	HELLO
ATTACK	LEMON	LXFOPV	ATTACK
ABC	KEY	KFA	ABC
HELLO WORLD	KEY	RIJVS UYVJN	HELLO WORLD

4.6.2 Result Analysis

The program successfully implements polyalphabetic substitution using the Vigenère Cipher. The key determines the substitution applied to each character, and decryption with the same key restores the original message.

Advantages

- Uses multiple substitution alphabets.
- Makes simple frequency analysis more difficult than monoalphabetic substitution.
- Easy to implement using a repeating keyword.
- Supports encryption and decryption of messages containing spaces and punctuation.

Issues / Limitations

- Repeating-key patterns can be discovered through cryptanalysis.
- Short keys may make the cipher easier to break.
- The sender and receiver must share the correct key.
- It is not suitable for modern confidential communication.

4.7 References

1. Course notes on Cryptography and Information Security.
2. William Stallings, Cryptography and Network Security: Principles and Practice.
3. Python 3 documentation: https://docs.python.org/3/

5. Quiz and Viva Questions

5.1 Quiz

Q1. Polyalphabetic substitution uses:

(a) Only one substitution alphabet (b) Multiple substitution alphabets (c) No key (d) Only binary values

Answer: (b) Multiple substitution alphabets.

Q2. Which of the following is a polyalphabetic cipher?

(a) Caesar Cipher (b) Vigenère Cipher (c) Rail Fence Cipher (d) Simple Columnar Transposition

Answer: (b) Vigenère Cipher.

Q3. Which formula represents Vigenère encryption?

(a) \(C=(P-K)\bmod26\) (b) \(C=(P+K)\bmod26\) (c) \(C=P\times K\) (d) \(C=P/K\)

Answer: (b) \(C=(P+K)\bmod26\).

Q4. What is the purpose of the keyword?

(a) To control the substitution applied to each character (b) To remove spaces (c) To sort letters (d) To calculate message length

Answer: (a) To control the substitution applied to each character.

Q5. Which cipher is generally more resistant to simple frequency analysis?

(a) Monoalphabetic substitution (b) Polyalphabetic substitution (c) Plaintext transmission (d) Simple character replacement with no key

Answer: (b) Polyalphabetic substitution.

5.2 Viva Questions with Answers

Q1. What is a Polyalphabetic Substitution Cipher?

It is a substitution cipher that uses multiple substitution alphabets to encrypt a message.

Q2. What is the Vigenère Cipher?

The Vigenère Cipher is a classical polyalphabetic substitution cipher that uses a keyword to determine the shift applied to each plaintext letter.

Q3. What is the difference between monoalphabetic and polyalphabetic substitution?

Monoalphabetic substitution uses one fixed substitution alphabet, while polyalphabetic substitution uses multiple substitution alphabets.

Q4. What is the role of the key in the Vigenère Cipher?

The key determines the shift applied to each plaintext character.

Q5. Why is the key repeated?

The key is repeated to provide a key character for every alphabetic character in the plaintext.

Q6. What is the encryption formula?

The encryption formula is \(C_i=(P_i+K_i)\bmod26\).

Q7. What is the decryption formula?

The decryption formula is \(P_i=(C_i-K_i+26)\bmod26\).

Q8. Why is modulo 26 used?

It ensures that alphabet positions remain between 0 and 25, wrapping around when necessary.

Q9. Does the Vigenère Cipher preserve spaces and punctuation?

In the program given here, spaces and punctuation are preserved and do not consume key characters.

Q10. Is the Vigenère Cipher secure today?

No. Repeating-key Vigenère implementations are vulnerable to classical cryptanalysis and should not be used for modern secure communication.

EXPERIMENT - 7

1. Title

Write a program to implement the Hill Cipher.

2. Outcome

Understand matrix-based encryption and decryption and implement the Hill Cipher using modular arithmetic and an invertible key matrix.

3. Objectives

- To understand the Hill Cipher and its mathematical foundation.
- To learn how matrices are used for encryption and decryption.
- To implement matrix multiplication under modulo 26.
- To understand the importance of an invertible key matrix.
- To verify encryption and decryption using test cases.

4. Nomenclature, Theory with Self-Assessment Questionnaire

4.1 Nomenclature

Term / Symbol	Meaning
Plaintext	Original readable message
Ciphertext	Encrypted message
Key matrix (K)	Matrix used for encryption
Inverse key matrix	Matrix used to reverse the encryption
Modulo 26	Arithmetic operation based on the 26 English letters
Determinant	Scalar value used to determine whether a matrix is invertible
Matrix multiplication	Mathematical operation used to transform plaintext blocks
Block	Group of characters encrypted together

4.2 Solution / Theory

Hill Cipher is a classical polygraphic substitution cipher developed by Lester S. Hill in 1929. It uses linear algebra and modular arithmetic to encrypt groups of letters.

Instead of encrypting every character independently, the Hill Cipher groups plaintext letters into blocks and represents each block as a numerical vector. The vector is multiplied by a key matrix, and the result is reduced modulo 26.

Each alphabet letter is represented numerically:

A = 0, B = 1, C = 2, ..., Z = 25.

For this experiment, a 2 × 2 key matrix is used, so the plaintext is processed in blocks of two letters.

Encryption formula:

$$ C=KP\bmod26 $$

Decryption formula: $$

P=K^{-1}C\bmod26 $$

Where:

- P is the plaintext vector.
- C is the ciphertext vector.
- K is the key matrix.
- K^{-1} is the modular inverse of the key matrix.

The key matrix must be invertible modulo 26. This means its determinant must be relatively prime to 26:

$$ \gcd(\det(K),26)=1 $$

If this condition is not satisfied, a valid modular inverse does not exist, and the matrix cannot be used for ordinary Hill Cipher decryption.

Example

Consider the key matrix:

$$ K= \begin{bmatrix} 3 & 3\\ 2 & 5 \end{bmatrix} $$

Plaintext: HELP

Divide the plaintext into blocks:

HE and LP

Convert the first block HE into numerical values:

- H = 7
- E = 4

Therefore,

$$ P= \begin{bmatrix} 7\\ 4 \end{bmatrix} $$

Multiply by the key matrix:

\[C= \begin{bmatrix} 3 & 3\\ 2 & 5 \end{bmatrix} \begin{bmatrix} 7\\ 4 \end{bmatrix} \bmod26 \]

\[C= \begin{bmatrix} 33\\ 34 \end{bmatrix} \bmod26 = \begin{bmatrix} 7\\ 8 \end{bmatrix} \]

The values 7 and 8 correspond to H and I.

Thus, HE becomes HI.

Similarly, the block LP becomes AT.

Therefore, the ciphertext is:

HIAT

Decryption uses the modular inverse of the key matrix to recover the original plaintext.

For this key matrix, the determinant is:

\[\det(K)=(3\times5)-(3\times2)=9 \]

Since \(\gcd(9,26)=1\), the matrix has a modular inverse and is valid for encryption and decryption.

4.3 Algorithm

Encryption

1. Start the program.
2. Define a valid 2 × 2 key matrix.
3. Read the plaintext message.
4. Convert the plaintext to uppercase and remove spaces.
5. If the number of letters is odd, append X.
6. Divide the message into blocks of two characters.
7. Convert each character into a numerical value from 0 to 25.
8. Multiply each plaintext vector by the key matrix.
9. Apply modulo 26 to each result.
10. Convert the numerical results back into letters.
11. Display the ciphertext.

Decryption

1. Calculate the determinant of the key matrix.
2. Find the modular inverse of the determinant modulo 26.

3. Calculate the inverse key matrix modulo 26.
4. Divide the ciphertext into blocks of two characters.
5. Convert each block into a numerical vector.
6. Multiply each vector by the inverse key matrix.
7. Apply modulo 26.
8. Convert the results back into letters.
9. Display the decrypted message.
10. Stop.

4.3 Dependencies •

Python 3

- No external libraries are

required. **4.4 Code / Pseudocode** from

math import gcd

```
def encrypt(text, key):
    text = ''.join(text.upper().split())

    if len(text) % 2 != 0:
        text += 'X'

    result = ""

    for i in range(0, len(text), 2):
a = ord(text[i]) - ord('A')         b =
ord(text[i + 1]) - ord('A')

        c1 = (key[0][0] * a + key[0][1] * b) % 26
c2 = (key[1][0] * a + key[1][1] * b) % 26
```

```
result += chr(c1 + ord('A'))        result += chr(c2
+ ord('A'))

    return result


def inverse_matrix(key):
    a, b = key[0]
c, d = key[1]

    det = (a * d - b * c) % 26

    if gcd(det, 26) != 1:
        raise ValueError("Key matrix is not invertible modulo 26")

    inv_det = pow(det, -1, 26)

    return [
        [(d * inv_det) % 26, (-b * inv_det) % 26],
        [(-c * inv_det) % 26, (a * inv_det) % 26]
    ]


def decrypt(text, key):
    inv_key = inverse_matrix(key)
result = ""

    for i in range(0, len(text), 2):
a = ord(text[i]) - ord('A')
```

```
        b = ord(text[i + 1]) - ord('A')

        p1 = (inv_key[0][0] * a +
inv_key[0][1] * b) % 26

        p2 = (inv_key[1][0] * a +
inv_key[1][1] * b) % 26

            result += chr(p1 + ord('A'))
        result += chr(p2 + ord('A'))

    return result


key = [
    [3, 3],
    [2, 5]
]

message = input("Enter plaintext: ")

encrypted = encrypt(message, key) print("Encrypted
text:", encrypted)

decrypted = decrypt(encrypted, key) print("Decrypted
text:", decrypted)
```

4.5 Results



4.5.1 Test Cases

Input Plaintext	Key Matrix	Expected Ciphertext	Decrypted Output
HELP	\(\begin{bmatrix}3&3\\2&5\end{bmatrix}\)	HIAT	HELP
ACT	Same matrix	RTD	ACTX
HELLO	Same matrix	HIOZHN	HELLOX

Note: The program adds X when the plaintext contains an odd number of letters. Therefore, the decrypted output may contain a trailing X that was added as padding.

4.5.2 Result Analysis

The program successfully implements the Hill Cipher using matrix multiplication and modular arithmetic. The key matrix transforms pairs of plaintext letters into ciphertext pairs. Decryption uses the modular inverse of the key matrix to recover the original message.

Advantages

- Encrypts groups of characters together.
- Uses mathematical operations and linear algebra.
- Makes direct single-letter substitution analysis more difficult.
- Demonstrates block-based classical encryption.

Issues / Limitations

- The key matrix must be invertible modulo 26.
- Incorrect key matrices cannot be used for ordinary decryption.

- The cipher is vulnerable to known-plaintext attacks.
- Padding may remain after decryption.
- It is not suitable for modern confidential communication.

4.6 References

1. Course notes on Cryptography and Information Security.
2. William Stallings, Cryptography and Network Security: Principles and Practice.
3. Python 3 documentation: https://docs.python.org/3/

5. Quiz and Viva Questions

5.1 Quiz

Q1. Hill Cipher is based on:

(a) Matrix multiplication (b) Sorting algorithms (c) File compression (d) Hash tables

Answer: (a) Matrix multiplication.

Q2. Which mathematical operation is used in Hill Cipher encryption?

(a) Ordinary division only (b) Modular arithmetic (c) Binary search (d) String sorting

Answer: (b) Modular arithmetic.

Q3. What is the condition for a 2 × 2 key matrix to be invertible modulo 26?

(a) Its determinant must be zero (b) Its determinant must be even (c) The determinant must be relatively prime to 26 (d) All matrix elements must be equal Answer: (c) The determinant must be relatively prime to 26.

Q4. Which formula represents Hill Cipher encryption? (a)

\(C=K+P\) (b) \(C=KP\bmod26\) (c) \(C=P/K\) (d) \(C=P^2\)

Answer: (b) \(C=KP\bmod26\).

Q5. What is required for Hill Cipher decryption?

(a) The inverse key matrix (b) Only the ciphertext length (c) A sorting algorithm (d) A random key for every character

Answer: (a) The inverse key matrix.

5.2 Viva Questions with Answers

Q1. What is the Hill Cipher?

Hill Cipher is a classical polygraphic substitution cipher that uses matrix multiplication and modular arithmetic to encrypt groups of characters.

Q2. Who developed the Hill Cipher?

Lester S. Hill developed the Hill Cipher in 1929.

Q3. What is a key matrix?

A key matrix is a square matrix used to transform plaintext vectors into ciphertext vectors.

Q4. Why is modulo 26 used?

Modulo 26 is used because the English alphabet contains 26 letters, represented numerically from 0 to 25.

Q5. What is the encryption formula for Hill Cipher?

The encryption formula is \(C=KP\bmod26\), where \(K\) is the key matrix and \(P\) is the plaintext vector.